



☆ Star 72.2k

Insights



-

[↑ Top](#)


```

6 #
7 # Licensed to PSF under a Contributor Agreement.
8
9 r"""Subprocesses with accessible I/O streams
10
11 This module allows you to spawn processes, connect to their
12 input/output/error pipes, and obtain their return codes.
13
14 For a complete description of this module see the Python documentation.
15
16 Main API
17 =====
18 run(...): Runs a command, waits for it to complete, then returns a
19     CompletedProcess instance.
20 Popen(...): A class for flexibly executing a command in a new process
21
22 Constants
23 -----
24 DEVNULL: Special value that indicates that os.devnull should be used
25 PIPE:    Special value that indicates a pipe should be created
26 STDOUT:  Special value that indicates that stderr should go to stdout
27
28
29 Older API
30 =====
31 call(...): Runs a command, waits for it to complete, then returns
32     the return code.
33 check_call(...): Same as call() but raises CalledProcessError()
34     if return code is not 0
35 check_output(...): Same as check_call() but returns the contents of
36     stdout instead of a return code
37 getoutput(...): Runs a command in the shell, waits for it to complete,
38     then returns the output
39 getstatusoutput(...): Runs a command in the shell, waits for it to complete,
40     then returns a (exitcode, output) tuple
41 """
42
43 import builtins
44 import errno
45 import io
46 import locale
47 import os
48 import time
49 import signal
50 import sys
51 import threading
52 import warnings
53 import contextlib
54 from time import monotonic as _time
55 import types
56
57 try:
58     import fcntl
59 except ImportError:
60     fcntl = None
61
62
63 __all__ = ["Popen", "PIPE", "STDOUT", "call", "check_call", "getstatusoutput",
64     "getoutput", "check_output", "run", "CalledProcessError", "DEVNULL",
65     "SubprocessError", "TimeoutExpired", "CompletedProcess"]
66
67 # NOTE: We intentionally exclude list2cmdline as it is

```

```
67         # considered an internal implementation detail.  issue10838.
68
69     # use presence of msvcrt to detect Windows-like platforms (see bpo-8110)
70     try:
71         import msvcrt
72     except ModuleNotFoundError:
73         _mswindows = False
74     else:
75         _mswindows = True
76
77     # some platforms do not support subprocesses
78     _can_fork_exec = sys.platform not in {"emscripten", "wasi", "ios", "tvos", "watchos"}
79
80     if _mswindows:
81         import _winapi
82         from _winapi import (CREATE_NEW_CONSOLE, CREATE_NEW_PROCESS_GROUP, # noqa: F401
83                               STD_INPUT_HANDLE, STD_OUTPUT_HANDLE,
84                               STD_ERROR_HANDLE, SW_HIDE,
85                               STARTF_USESTDHANDLES, STARTF_USESHOWWINDOW,
86                               STARTF_FORCEONFEEDBACK, STARTF_FORCEOFFFEEDBACK,
87                               ABOVE_NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS,
88                               HIGH_PRIORITY_CLASS, IDLE_PRIORITY_CLASS,
89                               NORMAL_PRIORITY_CLASS, REALTIME_PRIORITY_CLASS,
90                               CREATE_NO_WINDOW, DETACHED_PROCESS,
91                               CREATE_DEFAULT_ERROR_MODE, CREATE_BREAKAWAY_FROM_JOB)
92
93         __all__.extend(["CREATE_NEW_CONSOLE", "CREATE_NEW_PROCESS_GROUP",
94                         "STD_INPUT_HANDLE", "STD_OUTPUT_HANDLE",
95                         "STD_ERROR_HANDLE", "SW_HIDE",
96                         "STARTF_USESTDHANDLES", "STARTF_USESHOWWINDOW",
97                         "STARTF_FORCEONFEEDBACK", "STARTF_FORCEOFFFEEDBACK",
98                         "STARTUPINFO",
99                         "ABOVE_NORMAL_PRIORITY_CLASS", "BELOW_NORMAL_PRIORITY_CLASS",
100                        "HIGH_PRIORITY_CLASS", "IDLE_PRIORITY_CLASS",
101                        "NORMAL_PRIORITY_CLASS", "REALTIME_PRIORITY_CLASS",
102                        "CREATE_NO_WINDOW", "DETACHED_PROCESS",
103                        "CREATE_DEFAULT_ERROR_MODE", "CREATE_BREAKAWAY_FROM_JOB"])
104     else:
105         if _can_fork_exec:
106             from _posixsubprocess import fork_exec as _fork_exec
107             # used in methods that are called by __del__
108             class _del_safe:
109                 waitpid = os.waitpid
110                 waitstatus_to_exitcode = os.waitstatus_to_exitcode
111                 WIFSTOPPED = os.WIFSTOPPED
112                 WSTOPSIG = os.WSTOPSIG
113                 WNOHANG = os.WNOHANG
114                 ECHILD = errno.ECHILD
115         else:
116             class _del_safe:
117                 waitpid = None
118                 waitstatus_to_exitcode = None
119                 WIFSTOPPED = None
120                 WSTOPSIG = None
121                 WNOHANG = None
122                 ECHILD = errno.ECHILD
123
124         import select
125         import selectors
126
127
128     # Exception classes used by this module.
129     class SubprocessError(Exception): pass
130
131
132     class CalledProcessError(SubprocessError):
133         """Raised when run() is called with check=True and the process
134         returns a non-zero exit status.
135
136         Attributes:
137             cmd, returncode, stdout, stderr, output
138         """
139     def __init__(self, returncode, cmd, output=None, stderr=None):
140         self.returncode = returncode
141         self.cmd = cmd
142         self.output = output
143         self.stderr = stderr
144
145     def __str__(self):
146         if self.returncode and self.returncode < 0:
147             try:
148                 return "Command '%s' died with %r." % (
149                     self.cmd, signal.Signals(-self.returncode))
150             except ValueError:
151                 return "Command '%s' died with unknown signal %d." % (
152                     self.cmd, -self.returncode)
153         else:
154             return "Command '%s' returned non-zero exit status %d." % (
155                 self.cmd, self.returncode)
156
```

```

157     @property
158     def stdout(self):
159         """Alias for output attribute, to match stderr"""
160         return self.output
161
162     @stdout.setter
163     def stdout(self, value):
164         # There's no obvious reason to set this, but allow it anyway so
165         # .stdout is a transparent alias for .output
166         self.output = value
167
168
169     class TimeoutExpired(SubprocessError):
170         """This exception is raised when the timeout expires while waiting for a
171         child process.
172
173         Attributes:
174             cmd, output, stdout, stderr, timeout
175         """
176     def __init__(self, cmd, timeout, output=None, stderr=None):
177         self.cmd = cmd
178         self.timeout = timeout
179         self.output = output
180         self.stderr = stderr
181
182     def __str__(self):
183         return ("Command '%s' timed out after %s seconds" %
184                 (self.cmd, self.timeout))
185
186     @property
187     def stdout(self):
188         return self.output
189
190     @stdout.setter
191     def stdout(self, value):
192         # There's no obvious reason to set this, but allow it anyway so
193         # .stdout is a transparent alias for .output
194         self.output = value
195
196
197     if _mswindows:
198     class STARTUPINFO:
199     def __init__(self, *, dwFlags=0, hStdInput=None, hStdOutput=None,
200                 hStdError=None, wShowWindow=0, lpAttributeList=None):
201         self.dwFlags = dwFlags
202         self.hStdInput = hStdInput
203         self.hStdOutput = hStdOutput
204         self.hStdError = hStdError
205         self.wShowWindow = wShowWindow
206         self.lpAttributeList = lpAttributeList or {"handle_list": []}
207
208     def copy(self):
209         attr_list = self.lpAttributeList.copy()
210         if 'handle_list' in attr_list:
211             attr_list['handle_list'] = list(attr_list['handle_list'])
212
213         return STARTUPINFO(dwFlags=self.dwFlags,
214                             hStdInput=self.hStdInput,
215                             hStdOutput=self.hStdOutput,
216                             hStdError=self.hStdError,
217                             wShowWindow=self.wShowWindow,
218                             lpAttributeList=attr_list)
219
220
221     class Handle(int):
222         closed = False
223
224         def Close(self, CloseHandle=_winapi.CloseHandle):
225             if not self.closed:
226                 self.closed = True
227                 CloseHandle(self)
228
229     def Detach(self):
230         if not self.closed:
231             self.closed = True
232             return int(self)
233             raise ValueError("already closed")
234
235         def __repr__(self):
236             return "%s(%d)" % (self.__class__.__name__, int(self))
237
238         __del__ = Close
239     else:
240         # When select or poll has indicated that the file is writable,
241         # we can write up to _PIPE_BUF bytes without risk of blocking.
242         # POSIX defines PIPE_BUF as >= 512.
243         _PIPE_BUF = getattr(select, 'PIPE_BUF', 512)
244
245         # poll/select have the advantage of not requiring any extra file
246         # descriptor, contrarily to epoll/kqueue (also, they require a single
247         # syscall).

```

```
248         if hasattr(selectors, 'PollSelector'):
249             _PopenSelector = selectors.PollSelector
250         else:
251             _PopenSelector = selectors.SelectSelector
252
253
254     if _mswindows:
255         # On Windows we just need to close `Popen._handle` when we no longer need
256         # it, so that the kernel can free it. `Popen._handle` gets closed
257         # implicitly when the `Popen` instance is finalized (see `Handle.__del__`,
258         # which is calling `CloseHandle` as requested in [1]), so there is nothing
259         # for `_cleanup` to do.
260         #
261         # [1] https://docs.microsoft.com/en-us/windows/desktop/ProcThread/
262         # creating-processes
263         _active = None
264
265     def _cleanup():
266         pass
267     else:
268         # This lists holds Popen instances for which the underlying process had not
269         # exited at the time its __del__ method got called: those processes are
270         # wait()ed for synchronously from _cleanup() when a new Popen object is
271         # created, to avoid zombie processes.
272         _active = []
273
274     def _cleanup():
275         if _active is None:
276             return
277         for inst in _active[:]:
278             res = inst._internal_poll(_deadstate=sys.maxsize)
279             if res is not None:
280                 try:
281                     _active.remove(inst)
282                 except ValueError:
283                     # This can happen if two threads create a new Popen instance.
284                     # It's harmless that it was already removed, so ignore.
285                     pass
286
287     PIPE = -1
288     STDOUT = -2
289     DEVNULL = -3
290
291
292     # XXX This function is only used by multiprocessing and the test suite,
293     # but it's here so that it can be imported when Python is compiled without
294     # threads.
295
296     def _optim_args_from_interpreter_flags():
297         """Return a list of command-line arguments reproducing the current
298         optimization settings in sys.flags."""
299         args = []
300         value = sys.flags.optimize
301         if value > 0:
302             args.append('-' + '0' * value)
303         return args
304
305
306     def _args_from_interpreter_flags():
307         """Return a list of command-line arguments reproducing the current
308         settings in sys.flags, sys.warnoptions and sys.xoptions."""
309         flag_opt_map = {
310             'debug': 'd',
311             # 'inspect': 'i',
312             # 'interactive': 'i',
313             'dont_write_bytecode': 'B',
314             'no_site': 'S',
315             'verbose': 'v',
316             'bytes_warning': 'b',
317             'quiet': 'q',
318             # -O is handled in _optim_args_from_interpreter_flags()
319         }
320         args = _optim_args_from_interpreter_flags()
321         for flag, opt in flag_opt_map.items():
322             v = getattr(sys.flags, flag)
323             if v > 0:
324                 args.append('-' + opt * v)
325
326         if sys.flags.isolated:
327             args.append('-I')
328         else:
329             if sys.flags.ignore_environment:
330                 args.append('-E')
331             if sys.flags.no_user_site:
332                 args.append('-s')
333             if sys.flags.safe_path:
334                 args.append('-P')
335
336         # -W options
337         warnopts = sys.warnoptions[:]
```

```

338 xoptions = getattr(sys, '_xoptions', {})
339 bytes_warning = sys.flags.bytes_warning
340 dev_mode = sys.flags.dev_mode
341
342 if bytes_warning > 1:
343     warnopts.remove("error::BytesWarning")
344 elif bytes_warning:
345     warnopts.remove("default::BytesWarning")
346 if dev_mode:
347     warnopts.remove('default')
348 for opt in warnopts:
349     args.append('-W' + opt)
350
351 # -X options
352 if dev_mode:
353     args.extend(['-X', 'dev'])
354 for opt in sorted(xoptions):
355     if opt == 'dev':
356         # handled above via sys.flags.dev_mode
357         continue
358     value = xoptions[opt]
359     if value is True:
360         arg = opt
361     else:
362         arg = '%s=%s' % (opt, value)
363     args.extend(['-X', arg])
364
365     return args
366
367
368  def _text_encoding():
369     # Return default text encoding and emit EncodingWarning if
370     # sys.flags.warn_default_encoding is true.
371     if sys.flags.warn_default_encoding:
372         f = sys._getframe()
373         filename = f.f_code.co_filename
374         stacklevel = 2
375         while f := f.f_back:
376             if f.f_code.co_filename != filename:
377                 break
378             stacklevel += 1
379         warnings.warn("'encoding' argument not specified.",
380                       EncodingWarning, stacklevel)
381
382     if sys.flags.utf8_mode:
383         return "utf-8"
384     else:
385         return locale.getencoding()
386
387
388  def call(*popenargs, timeout=None, **kwargs):
389     """Run command with arguments.  Wait for command to complete or
390     for timeout seconds, then return the returncode attribute.
391
392     The arguments are the same as for the Popen constructor.  Example:
393
394     retcode = call(["ls", "-l"])
395     """
396     with Popen(*popenargs, **kwargs) as p:
397         try:
398             return p.wait(timeout=timeout)
399         except: # Including KeyboardInterrupt, wait handled that.
400             p.kill()
401             # We don't call p.wait() again as p.__exit__ does that for us.
402             raise
403
404
405  def check_call(*popenargs, **kwargs):
406     """Run command with arguments.  Wait for command to complete.  If
407     the exit code was zero then return, otherwise raise
408     CalledProcessError.  The CalledProcessError object will have the
409     return code in the returncode attribute.
410
411     The arguments are the same as for the call function.  Example:
412
413     check_call(["ls", "-l"])
414     """
415     retcode = call(*popenargs, **kwargs)
416     if retcode:
417         cmd = kwargs.get("args")
418         if cmd is None:
419             cmd = popenargs[0]
420         raise CalledProcessError(retcode, cmd)
421     return 0
422
423
424  def check_output(*popenargs, timeout=None, **kwargs):
425     r"""Run command with arguments and return its output.
426
427     If the exit code was non-zero it raises a CalledProcessError.  The
428     CalledProcessError object will have the return code in the returncode

```

```

429         called process object to have the return code in the returncode
430         attribute and output in the output attribute.
431
432     The arguments are the same as for the Popen constructor.  Example:
433
434     >>> check_output(["ls", "-l", "/dev/null"])
435     b'crw-rw-rw- 1 root root 1, 3 Oct 18  2007 /dev/null\n'
436
437     The stdout argument is not allowed as it is used internally.
438     To capture standard error in the result, use stderr=STDOUT.
439
440     >>> check_output(["/bin/sh", "-c",
441     ...               "ls -l non_existent_file ; exit 0"],
442     ...               stderr=STDOUT)
443     b'ls: non_existent_file: No such file or directory\n'
444
445     There is an additional optional argument, "input", allowing you to
446     pass a string to the subprocess's stdin.  If you use this argument
447     you may not also use the Popen constructor's "stdin" argument, as
448     it too will be used internally.  Example:
449
450     >>> check_output(["sed", "-e", "s/foo/bar/"],
451     ...               input=b"when in the course of fooman events\n")
452     b'when in the course of barman events\n'
453
454     By default, all communication is in bytes, and therefore any "input"
455     should be bytes, and the return value will be bytes.  If in text mode,
456     any "input" should be a string, and the return value will be a string
457     decoded according to locale encoding, or by "encoding" if set. Text mode
458     is triggered by setting any of text, encoding, errors or universal_newlines.
459     """
460     for kw in ('stdout', 'check'):
461         if kw in kwargs:
462             raise ValueError(f'{kw} argument not allowed, it will be overridden.')
463
464     if 'input' in kwargs and kwargs['input'] is None:
465         # Explicitly passing input=None was previously equivalent to passing an
466         # empty string. That is maintained here for backwards compatibility.
467         if kwargs.get('universal_newlines') or kwargs.get('text') or kwargs.get('encoding') \
468            or kwargs.get('errors'):
469             empty = ''
470         else:
471             empty = b''
472         kwargs['input'] = empty
473
474     return run(*popenargs, stdout=PIPE, timeout=timeout, check=True,
475               **kwargs).stdout
476
477 class CompletedProcess(object):
478     """A process that has finished running.
479
480     This is returned by run().
481
482     Attributes:
483         args: The list or str args passed to run().
484         returncode: The exit code of the process, negative for signals.
485         stdout: The standard output (None if not captured).
486         stderr: The standard error (None if not captured).
487     """
488     def __init__(self, args, returncode, stdout=None, stderr=None):
489         self.args = args
490         self.returncode = returncode
491         self.stdout = stdout
492         self.stderr = stderr
493
494     def __repr__(self):
495         args = ['args={!r}'.format(self.args),
496                'returncode={!r}'.format(self.returncode)]
497         if self.stdout is not None:
498             args.append('stdout={!r}'.format(self.stdout))
499         if self.stderr is not None:
500             args.append('stderr={!r}'.format(self.stderr))
501         return "{}({})".format(type(self).__name__, ', '.join(args))
502
503     __class_getitem__ = classmethod(types.GenericAlias)
504
505     def check_returncode(self):
506         """Raise CalledProcessError if the exit code is non-zero."""
507         if self.returncode:
508             raise CalledProcessError(self.returncode, self.args, self.stdout,
509                                     self.stderr)
510
511
512
513 def run(*popenargs,
514        input=None, capture_output=False, timeout=None, check=False, **kwargs):
515     """Run command with arguments and return a CompletedProcess instance.
516
517     The returned instance will have attributes args, returncode, stdout and
518     stderr. By default, stdout and stderr are not captured, and those attributes

```

```
519 will be None. Pass stdout=PIPE and/or stderr=PIPE in order to capture them,
520 or pass capture_output=True to capture both.
521
522 If check is True and the exit code was non-zero, it raises a
523 CalledProcessError. The CalledProcessError object will have the return code
524 in the returncode attribute, and output & stderr attributes if those streams
525 were captured.
526
527 If timeout (seconds) is given and the process takes too long,
528 a TimeoutExpired exception will be raised.
529
530 There is an optional argument "input", allowing you to
531 pass bytes or a string to the subprocess's stdin. If you use this argument
532 you may not also use the Popen constructor's "stdin" argument, as
533 it will be used internally.
534
535 By default, all communication is in bytes, and therefore any "input" should
536 be bytes, and the stdout and stderr will be bytes. If in text mode, any
537 "input" should be a string, and stdout and stderr will be strings decoded
538 according to locale encoding, or by "encoding" if set. Text mode is
539 triggered by setting any of text, encoding, errors or universal_newlines.
540
541 The other arguments are the same as for the Popen constructor.
542 """
543 if input is not None:
544     if kwargs.get('stdin') is not None:
545         raise ValueError('stdin and input arguments may not both be used.')
546     kwargs['stdin'] = PIPE
547
548 if capture_output:
549     if kwargs.get('stdout') is not None or kwargs.get('stderr') is not None:
550         raise ValueError('stdout and stderr arguments may not be used '
551                            'with capture_output.')
552     kwargs['stdout'] = PIPE
553     kwargs['stderr'] = PIPE
554
555 with Popen(*popenargs, **kwargs) as process:
556     try:
557         stdout, stderr = process.communicate(input, timeout=timeout)
558     except TimeoutExpired as exc:
559         process.kill()
560         if _mswindows:
561             # Windows accumulates the output in a single blocking
562             # read() call run on child threads, with the timeout
563             # being done in a join() on those threads. communicate()
564             # _after_ kill() is required to collect that and add it
565             # to the exception.
566             exc.stdout, exc.stderr = process.communicate()
567         else:
568             # POSIX _communicate already populated the output so
569             # far into the TimeoutExpired exception.
570             process.wait()
571         raise
572     except: # Including KeyboardInterrupt, communicate handled that.
573         process.kill()
574         # We don't call process.wait() as __exit__ does that for us.
575         raise
576     retcode = process.poll()
577     if check and retcode:
578         raise CalledProcessError(retcode, process.args,
579                                output=stdout, stderr=stderr)
580 return CompletedProcess(process.args, retcode, stdout, stderr)
581
582
583  ✓ def list2cmdline(seq):
584     """
585     Translate a sequence of arguments into a command line
586     string, using the same rules as the MS C runtime:
587
588     1) Arguments are delimited by white space, which is either a
589        space or a tab.
590
591     2) A string surrounded by double quotation marks is
592        interpreted as a single argument, regardless of white space
593        contained within. A quoted string can be embedded in an
594        argument.
595
596     3) A double quotation mark preceded by a backslash is
597        interpreted as a literal double quotation mark.
598
599     4) Backslashes are interpreted literally, unless they
600        immediately precede a double quotation mark.
601
602     5) If backslashes immediately precede a double quotation mark,
603        every pair of backslashes is interpreted as a literal
604        backslash. If the number of backslashes is odd, the last
605        backslash escapes the next double quotation mark as
606        described in rule 3.
607     """
608
609     # See
```

```

609 # See
610 # http://msdn.microsoft.com/en-us/library/17w5ykft.aspx
611 # or search http://msdn.microsoft.com for
612 # "Parsing C++ Command-Line Arguments"
613 result = []
614 needquote = False
615 for arg in map(os.fsdecode, seq):
616     bs_buf = []
617
618     # Add a space to separate this argument from the others
619     if result:
620         result.append(' ')
621
622     needquote = (" " in arg) or ("\t" in arg) or not arg
623     if needquote:
624         result.append('')
625
626     for c in arg:
627         if c == '\\':
628             # Don't know if we need to double yet.
629             bs_buf.append(c)
630         elif c == '"':
631             # Double backslashes.
632             result.append('\\' * len(bs_buf)*2)
633             bs_buf = []
634             result.append('\\"')
635         else:
636             # Normal char
637             if bs_buf:
638                 result.extend(bs_buf)
639                 bs_buf = []
640             result.append(c)
641
642     # Add remaining backslashes, if any.
643     if bs_buf:
644         result.extend(bs_buf)
645
646     if needquote:
647         result.extend(bs_buf)
648         result.append('')
649
650     return ''.join(result)
651
652
653 # Various tools for executing commands and looking at their output and status.
654 #
655
656 def getstatusoutput(cmd, *, encoding=None, errors=None):
657     """Return (exitcode, output) of executing cmd in a shell.
658
659     Execute the string 'cmd' in a shell with 'check_output' and
660     return a 2-tuple (status, output). The locale encoding is used
661     to decode the output and process newlines.
662
663     A trailing newline is stripped from the output.
664     The exit status for the command can be interpreted
665     according to the rules for the function 'wait'. Example:
666
667     >>> import subprocess
668     >>> subprocess.getstatusoutput('ls /bin/ls')
669     (0, '/bin/ls')
670     >>> subprocess.getstatusoutput('cat /bin/junk')
671     (1, 'cat: /bin/junk: No such file or directory')
672     >>> subprocess.getstatusoutput('/bin/junk')
673     (127, 'sh: /bin/junk: not found')
674     >>> subprocess.getstatusoutput('/bin/kill $$')
675     (-15, '')
676     """
677     try:
678         data = check_output(cmd, shell=True, text=True, stderr=STDOUT,
679                             encoding=encoding, errors=errors)
680
681         exitcode = 0
682     except CalledProcessError as ex:
683         data = ex.output
684         exitcode = ex.returncode
685     if data[-1:] == '\n':
686         data = data[:-1]
687     return exitcode, data
688
689 def getoutput(cmd, *, encoding=None, errors=None):
690     """Return output (stdout or stderr) of executing cmd in a shell.
691
692     Like getstatusoutput(), except the exit status is ignored and the return
693     value is a string containing the command's output. Example:
694
695     >>> import subprocess
696     >>> subprocess.getoutput('ls /bin/ls')
697     '/bin/ls'
698     """
699     return getstatusoutput(cmd, encoding=encoding, errors=errors)[1]

```

```
700
701
702  ✓ def _use_posix_spawn():
703     """Check if posix_spawn() can be used for subprocess.
704
705     subprocess requires a posix_spawn() implementation that properly reports
706     errors to the parent process, & sets errno on the following failures:
707
708     * Process attribute actions failed.
709     * File actions failed.
710     * exec() failed.
711
712     Prefer an implementation which can use vfork() in some cases for best
713     performance.
714     """
715     if _mswindows or not hasattr(os, 'posix_spawn'):
716         # os.posix_spawn() is not available
717         return False
718
719     if ((_env := os.environ.get('_PYTHON_SUBPROCESS_USE_POSIX_SPAWN')) in ('0', '1')):
720         return bool(int(_env))
721
722     if sys.platform in ('darwin', 'sunos5'):
723         # posix_spawn() is a syscall on both macOS and Solaris,
724         # and properly reports errors
725         return True
726
727     # Check libc name and runtime libc version
728     try:
729         ver = os.confstr('CS_GNU_LIBC_VERSION')
730         # parse 'glibc 2.28' as ('glibc', (2, 28))
731         parts = ver.split(maxsplit=1)
732         if len(parts) != 2:
733             # reject unknown format
734             raise ValueError
735         libc = parts[0]
736         version = tuple(map(int, parts[1].split('.')))
737
738         if sys.platform == 'linux' and libc == 'glibc' and version >= (2, 24):
739             # glibc 2.24 has a new Linux posix_spawn implementation using vfork
740             # which properly reports errors to the parent process.
741             return True
742
743             # Note: Don't use the implementation in earlier glibc because it doesn't
744             # use vfork (even if glibc 2.26 added a pipe to properly report errors
745             # to the parent process).
746     except (AttributeError, ValueError, OSError):
747         # os.confstr() or CS_GNU_LIBC_VERSION value not available
748         pass
749
750     # By default, assume that posix_spawn() does not properly report errors.
751     return False
752
753 # These are primarily fail-safe knobs for negatives. A True value does not
754 # guarantee the given libc/syscall API will be used.
755 _USE_POSIX_SPAWN = _use_posix_spawn()
756 _HAVE_POSIX_SPAWN_CLOSEFROM = hasattr(os, 'POSIX_SPAWN_CLOSEFROM')
757
758
759  ✓ class Popen:
760     """ Execute a child program in a new process.
761
762     For a complete description of the arguments see the Python documentation.
763
764     Arguments:
765         args: A string, or a sequence of program arguments.
766
767         bufsize: supplied as the buffering argument to the open() function when
768                 creating the stdin/stdout/stderr pipe file objects
769
770         executable: A replacement program to execute.
771
772         stdin, stdout and stderr: These specify the executed programs' standard
773                                 input, standard output and standard error file handles, respectively.
774
775         preexec_fn: (POSIX only) An object to be called in the child process
776                     just before the child is executed.
777
778         close_fds: Controls closing or inheriting of file descriptors.
779
780         shell: If true, the command will be executed through the shell.
781
782         cwd: Sets the current directory before the child is executed.
783
784         env: Defines the environment variables for the new process.
785
786         text: If true, decode stdin, stdout and stderr using the given encoding
787              (if set) or the system default otherwise.
788
789         universal_newlines: Alias of text, provided for backwards compatibility.
790
```

```
790
791     startupinfo and creationflags (Windows only)
792
793     restore_signals (POSIX only)
794
795     start_new_session (POSIX only)
796
797     process_group (POSIX only)
798
799     group (POSIX only)
800
801     extra_groups (POSIX only)
802
803     user (POSIX only)
804
805     umask (POSIX only)
806
807     pass_fds (POSIX only)
808
809     encoding and errors: Text mode encoding and error handling to use for
810         file objects stdin, stdout and stderr.
811
812     Attributes:
813         stdin, stdout, stderr, pid, returncode
814     """
815     _child_created = False # Set here since __del__ checks it
816
817     def __init__(self, args, bufsize=-1, executable=None,
818                 stdin=None, stdout=None, stderr=None,
819                 preexec_fn=None, close_fds=True,
820                 shell=False, cwd=None, env=None, universal_newlines=None,
821                 startupinfo=None, creationflags=0,
822                 restore_signals=True, start_new_session=False,
823                 pass_fds=(), *, user=None, group=None, extra_groups=None,
824                 encoding=None, errors=None, text=None, umask=-1, pipesize=-1,
825                 process_group=None):
826         """Create new Popen instance."""
827         if not _can_fork_exec:
828             raise OSError(
829                 errno.ENOTSUP, f"{sys.platform} does not support processes."
830             )
831
832         _cleanup()
833         # Held while anything is calling waitpid before returncode has been
834         # updated to prevent clobbering returncode if wait() or poll() are
835         # called from multiple threads at once. After acquiring the lock,
836         # code must re-check self.returncode to see if another thread just
837         # finished a waitpid() call.
838         self._waitpid_lock = threading.Lock()
839
840         self._input = None
841         self._communication_started = False
842         if bufsize is None:
843             bufsize = -1 # Restore default
844         if not isinstance(bufsize, int):
845             raise TypeError("bufsize must be an integer")
846
847         if stdout is STDOUT:
848             raise ValueError("STDOUT can only be used for stderr")
849
850         if pipesize is None:
851             pipesize = -1 # Restore default
852         if not isinstance(pipesize, int):
853             raise TypeError("pipesize must be an integer")
854
855         if _mswindows:
856             if preexec_fn is not None:
857                 raise ValueError("preexec_fn is not supported on Windows "
858                                 "platforms")
859         else:
860             # POSIX
861             if pass_fds and not close_fds:
862                 warnings.warn("pass_fds overriding close_fds.", RuntimeWarning)
863                 close_fds = True
864             if startupinfo is not None:
865                 raise ValueError("startupinfo is only supported on Windows "
866                                 "platforms")
867             if creationflags != 0:
868                 raise ValueError("creationflags is only supported on Windows "
869                                 "platforms")
870
871         self.args = args
872         self.stdin = None
873         self.stdout = None
874         self.stderr = None
875         self.pid = None
876         self.returncode = None
877         self.encoding = encoding
878         self.errors = errors
879         self.pipesize = pipesize
880
```

```
881 # Validate the combinations of text and universal_newlines
882 if (text is not None and universal_newlines is not None
883     and bool(universal_newlines) != bool(text)):
884     raise SubprocessError('Cannot disambiguate when both text '
885                           'and universal_newlines are supplied but '
886                           'different. Pass one or the other.')
887
888 self.text_mode = encoding or errors or text or universal_newlines
889 if self.text_mode and encoding is None:
890     self.encoding = encoding = _text_encoding()
891
892 # How long to resume waiting on a child after the first ^C.
893 # There is no right value for this. The purpose is to be polite
894 # yet remain good for interactive users trying to exit a tool.
895 self._sigint_wait_secs = 0.25 # 1/xkcd221.getRandomNumber()
896
897 self._closed_child_pipe_fds = False
898
899 if self.text_mode:
900     if bufsize == 1:
901         line_buffering = True
902         # Use the default buffer size for the underlying binary streams
903         # since they don't support line buffering.
904         bufsize = -1
905     else:
906         line_buffering = False
907
908 if process_group is None:
909     process_group = -1 # The internal APIs are int-only
910
911 gid = None
912 if group is not None:
913     if not hasattr(os, 'setregid'):
914         raise ValueError("The 'group' parameter is not supported on the "
915                           "current platform")
916
917     elif isinstance(group, str):
918         try:
919             import grp
920         except ImportError:
921             raise ValueError("The group parameter cannot be a string "
922                               "on systems without the grp module")
923
924         gid = grp.getgrnam(group).gr_gid
925     elif isinstance(group, int):
926         gid = group
927     else:
928         raise TypeError("Group must be a string or an integer, not {}".
929                           .format(type(group)))
930
931     if gid < 0:
932         raise ValueError(f"Group ID cannot be negative, got {gid}")
933
934 gids = None
935 if extra_groups is not None:
936     if not hasattr(os, 'setgroups'):
937         raise ValueError("The 'extra_groups' parameter is not "
938                           "supported on the current platform")
939
940     elif isinstance(extra_groups, str):
941         raise ValueError("Groups must be a list, not a string")
942
943     gids = []
944     for extra_group in extra_groups:
945         if isinstance(extra_group, str):
946             try:
947                 import grp
948             except ImportError:
949                 raise ValueError("Items in extra_groups cannot be "
950                                   "strings on systems without the "
951                                   "grp module")
952
953             gids.append(grp.getgrnam(extra_group).gr_gid)
954         elif isinstance(extra_group, int):
955             gids.append(extra_group)
956         else:
957             raise TypeError("Items in extra_groups must be a string "
958                               "or integer, not {}".
959                               .format(type(extra_group)))
960
961     # make sure that the gids are all positive here so we can do less
962     # checking in the C code
963     for gid_check in gids:
964         if gid_check < 0:
965             raise ValueError(f"Group ID cannot be negative, got {gid_check}")
966
967 uid = None
968 if user is not None:
969     if not hasattr(os, 'setreuid'):
970         raise ValueError("The 'user' parameter is not supported on "
```

```
971         the current platform")
972
973     elif isinstance(user, str):
974         try:
975             import pwd
976         except ImportError:
977             raise ValueError("The user parameter cannot be a string "
978                             "on systems without the pwd module")
979         uid = pwd.getpwnam(user).pw_uid
980     elif isinstance(user, int):
981         uid = user
982     else:
983         raise TypeError("User must be a string or an integer")
984
985     if uid < 0:
986         raise ValueError(f"User ID cannot be negative, got {uid}")
987
988 # Input and output objects. The general principle is like
989 # this:
990 #
991 # Parent                Child
992 # -----              -
993 # p2cwrite  ---stdin--> p2cread
994 # c2pread   <---stdout--- c2pwrite
995 # errread   <---stderr--- errwrite
996 #
997 # On POSIX, the child objects are file descriptors.  On
998 # Windows, these are Windows file handles.  The parent objects
999 # are file descriptors on both platforms.  The parent objects
1000 # are -1 when not using PIPEs. The child objects are -1
1001 # when not redirecting.
1002
1003 (p2cread, p2cwrite,
1004  c2pread, c2pwrite,
1005  errread, errwrite) = self._get_handles(stdin, stdout, stderr)
1006
1007 # From here on, raising exceptions may cause file descriptor leakage
1008
1009 # We wrap OS handles *before* launching the child, otherwise a
1010 # quickly terminating child could make our fds unwrappable
1011 # (see #8458).
1012
1013 if _mswindows:
1014     if p2cwrite != -1:
1015         p2cwrite = msvcrt.open_osfhandle(p2cwrite.Detach(), 0)
1016     if c2pread != -1:
1017         c2pread = msvcrt.open_osfhandle(c2pread.Detach(), 0)
1018     if errread != -1:
1019         errread = msvcrt.open_osfhandle(errread.Detach(), 0)
1020
1021 try:
1022     if p2cwrite != -1:
1023         self.stdin = io.open(p2cwrite, 'wb', bufsize)
1024         if self.text_mode:
1025             self.stdin = io.TextIOWrapper(self.stdin, write_through=True,
1026                                           line_buffering=line_buffering,
1027                                           encoding=encoding, errors=errors)
1028     if c2pread != -1:
1029         self.stdout = io.open(c2pread, 'rb', bufsize)
1030         if self.text_mode:
1031             self.stdout = io.TextIOWrapper(self.stdout,
1032                                           encoding=encoding, errors=errors)
1033     if errread != -1:
1034         self.stderr = io.open(errread, 'rb', bufsize)
1035         if self.text_mode:
1036             self.stderr = io.TextIOWrapper(self.stderr,
1037                                           encoding=encoding, errors=errors)
1038
1039     self._execute_child(args, executable, preexec_fn, close_fds,
1040                        pass_fds, cwd, env,
1041                        startupinfo, creationflags, shell,
1042                        p2cread, p2cwrite,
1043                        c2pread, c2pwrite,
1044                        errread, errwrite,
1045                        restore_signals,
1046                        gid, gids, uid, umask,
1047                        start_new_session, process_group)
1048 except:
1049     # Cleanup if the child failed starting.
1050     for f in filter(None, (self.stdin, self.stdout, self.stderr)):
1051         try:
1052             f.close()
1053         except OSError:
1054             pass # Ignore EBADF or other errors.
1055
1056 if not self._closed_child_pipe_fds:
1057     to_close = []
1058     if stdin == PIPE:
1059         to_close.append(p2cread)
1060     if stdout == PIPE:
1061         to_close.append(c2pwrite)
```

```
1062         if stderr == PIPE:
1063             to_close.append(errwrite)
1064         if hasattr(self, '_devnull'):
1065             to_close.append(self._devnull)
1066         for fd in to_close:
1067             try:
1068                 if _mswindows and isinstance(fd, Handle):
1069                     fd.Close()
1070             else:
1071                 os.close(fd)
1072         except OSError:
1073             pass
1074
1075         raise
1076
1077     def __repr__(self):
1078         obj_repr = (
1079             f"<{self.__class__.__name__}: "
1080             f"returncode: {self.returncode} args: {self.args!r}>"
1081         )
1082         if len(obj_repr) > 80:
1083             obj_repr = obj_repr[:76] + "...>"
1084         return obj_repr
1085
1086     __class_getitem__ = classmethod(types.GenericAlias)
1087
1088     @property
1089     def universal_newlines(self):
1090         # universal_newlines as retained as an alias of text_mode for API
1091         # compatibility. bpo-31756
1092         return self.text_mode
1093
1094     @universal_newlines.setter
1095     def universal_newlines(self, universal_newlines):
1096         self.text_mode = bool(universal_newlines)
1097
1098     def _translate_newlines(self, data, encoding, errors):
1099         data = data.decode(encoding, errors)
1100         return data.replace("\r\n", "\n").replace("\r", "\n")
1101
1102     def __enter__(self):
1103         return self
1104
1105     def __exit__(self, exc_type, value, traceback):
1106         if self.stdout:
1107             self.stdout.close()
1108         if self.stderr:
1109             self.stderr.close()
1110         try: # Flushing a BufferedWriter may raise an error
1111             if self.stdin:
1112                 self.stdin.close()
1113         finally:
1114             if exc_type == KeyboardInterrupt:
1115                 # https://bugs.python.org/issue25942
1116                 # In the case of a KeyboardInterrupt we assume the SIGINT
1117                 # was also already sent to our child processes. We can't
1118                 # block indefinitely as that is not user friendly.
1119                 # If we have not already waited a brief amount of time in
1120                 # an interrupted .wait() or .communicate() call, do so here
1121                 # for consistency.
1122                 if self._sigint_wait_secs > 0:
1123                     try:
1124                         self._wait(timeout=self._sigint_wait_secs)
1125                     except TimeoutExpired:
1126                         pass
1127                 self._sigint_wait_secs = 0 # Note that this has been done.
1128             else:
1129                 # Wait for the process to terminate, to avoid zombies.
1130                 self.wait()
1131
1132     def __del__(self, _maxsize=sys.maxsize, _warn=warnings.warn):
1133         if not self._child_created:
1134             # We didn't get to successfully create a child process.
1135             return
1136         if self.returncode is None:
1137             # Not reading subprocess exit status creates a zombie process which
1138             # is only destroyed at the parent python process exit
1139             _warn("subprocess %s is still running" % self.pid,
1140                  ResourceWarning, source=self)
1141         # In case the child hasn't been waited on, check if it's done.
1142         self._internal_poll(_deadstate=_maxsize)
1143         if self.returncode is None and _active is not None:
1144             # Child is still running, keep us alive until we can wait on it.
1145             _active.append(self)
1146
1147     def _get_devnull(self):
1148         if not hasattr(self, '_devnull'):
1149             self._devnull = os.open(os.devnull, os.O_RDWR)
1150         return self._devnull
1151
```

```

1152 def _stdin_write(self, input):
1153     if input:
1154         try:
1155             self.stdin.write(input)
1156         except BrokenPipeError:
1157             pass # communicate() must ignore broken pipe errors.
1158         except OSError as exc:
1159             if exc.errno == errno.EINVAL:
1160                 # bpo-19612, bpo-30418: On Windows, stdin.write() fails
1161                 # with EINVAL if the child process exited or if the child
1162                 # process is still running but closed the pipe.
1163                 pass
1164             else:
1165                 raise
1166
1167     try:
1168         self.stdin.close()
1169     except BrokenPipeError:
1170         pass # communicate() must ignore broken pipe errors.
1171     except OSError as exc:
1172         if exc.errno == errno.EINVAL:
1173             pass
1174         else:
1175             raise
1176
1177 def communicate(self, input=None, timeout=None):
1178     """Interact with process: Send data to stdin and close it.
1179     Read data from stdout and stderr, until end-of-file is
1180     reached. Wait for process to terminate.
1181
1182     The optional "input" argument should be data to be sent to the
1183     child process, or None, if no data should be sent to the child.
1184     communicate() returns a tuple (stdout, stderr).
1185
1186     By default, all communication is in bytes, and therefore any
1187     "input" should be bytes, and the (stdout, stderr) will be bytes.
1188     If in text mode (indicated by self.text_mode), any "input" should
1189     be a string, and (stdout, stderr) will be strings decoded
1190     according to locale encoding, or by "encoding" if set. Text mode
1191     is triggered by setting any of text, encoding, errors or
1192     universal_newlines.
1193     """
1194
1195     if self._communication_started and input:
1196         raise ValueError("Cannot send input after starting communication")
1197
1198     # Optimization: If we are not worried about timeouts, we haven't
1199     # started communicating, and we have one or zero pipes, using select()
1200     # or threads is unnecessary.
1201     if (timeout is None and not self._communication_started and
1202         [self.stdin, self.stdout, self.stderr].count(None) >= 2):
1203         stdout = None
1204         stderr = None
1205         if self.stdin:
1206             self._stdin_write(input)
1207         elif self.stdout:
1208             stdout = self.stdout.read()
1209             self.stdout.close()
1210         elif self.stderr:
1211             stderr = self.stderr.read()
1212             self.stderr.close()
1213         self.wait()
1214     else:
1215         if timeout is not None:
1216             endtime = _time() + timeout
1217         else:
1218             endtime = None
1219
1220         try:
1221             stdout, stderr = self._communicate(input, endtime, timeout)
1222         except KeyboardInterrupt:
1223             # https://bugs.python.org/issue25942
1224             # See the detailed comment in .wait().
1225             if timeout is not None:
1226                 sigint_timeout = min(self._sigint_wait_secs,
1227                                     self._remaining_time(endtime))
1228             else:
1229                 sigint_timeout = self._sigint_wait_secs
1230             self._sigint_wait_secs = 0 # nothing else should wait.
1231             try:
1232                 self._wait(timeout=sigint_timeout)
1233             except TimeoutExpired:
1234                 pass
1235             raise # resume the KeyboardInterrupt
1236
1237         finally:
1238             self._communication_started = True
1239     try:
1240         sts = self.wait(timeout=self._remaining_time(endtime))
1241     except TimeoutExpired as exc:
1242         exc.timeout = timeout

```

```
1243         raise
1244
1245     return (stdout, stderr)
1246
1247
1248     def poll(self):
1249         """Check if child process has terminated. Set and return returncode
1250         attribute."""
1251         return self._internal_poll()
1252
1253
1254     def _remaining_time(self, endtime):
1255         """Convenience for _communicate when computing timeouts."""
1256         if endtime is None:
1257             return None
1258         else:
1259             return endtime - _time()
1260
1261
1262     def _check_timeout(self, endtime, orig_timeout, stdout_seq, stderr_seq,
1263                       skip_check_and_raise=False):
1264         """Convenience for checking if a timeout has expired."""
1265         if endtime is None:
1266             return
1267         if skip_check_and_raise or _time() > endtime:
1268             raise TimeoutExpired(
1269                 self.args, orig_timeout,
1270                 output=b''.join(stdout_seq) if stdout_seq else None,
1271                 stderr=b''.join(stderr_seq) if stderr_seq else None)
1272
1273
1274     def wait(self, timeout=None):
1275         """Wait for child process to terminate; returns self.returncode."""
1276         if timeout is not None:
1277             endtime = _time() + timeout
1278         try:
1279             return self._wait(timeout=timeout)
1280         except KeyboardInterrupt:
1281             # https://bugs.python.org/issue25942
1282             # The first keyboard interrupt waits briefly for the child to
1283             # exit under the common assumption that it also received the ^C
1284             # generated SIGINT and will exit rapidly.
1285             if timeout is not None:
1286                 sigint_timeout = min(self._sigint_wait_secs,
1287                                     self._remaining_time(endtime))
1288             else:
1289                 sigint_timeout = self._sigint_wait_secs
1290             self._sigint_wait_secs = 0 # nothing else should wait.
1291             try:
1292                 self._wait(timeout=sigint_timeout)
1293             except TimeoutExpired:
1294                 pass
1295             raise # resume the KeyboardInterrupt
1296
1297     def _close_pipe_fds(self,
1298                       p2cread, p2cwrite,
1299                       c2pread, c2pwrite,
1300                       errread, errwrite):
1301         # self._devnull is not always defined.
1302         devnull_fd = getattr(self, '_devnull', None)
1303
1304         with contextlib.ExitStack() as stack:
1305             if _mswindows:
1306                 if p2cread != -1:
1307                     stack.callback(p2cread.Close)
1308                 if c2pwrite != -1:
1309                     stack.callback(c2pwrite.Close)
1310                 if errwrite != -1:
1311                     stack.callback(errwrite.Close)
1312             else:
1313                 if p2cread != -1 and p2cwrite != -1 and p2cread != devnull_fd:
1314                     stack.callback(os.close, p2cread)
1315                 if c2pwrite != -1 and c2pread != -1 and c2pwrite != devnull_fd:
1316                     stack.callback(os.close, c2pwrite)
1317                 if errwrite != -1 and errread != -1 and errwrite != devnull_fd:
1318                     stack.callback(os.close, errwrite)
1319
1320             if devnull_fd is not None:
1321                 stack.callback(os.close, devnull_fd)
1322
1323         # Prevent a double close of these handles/fds from __init__ on error.
1324         self._closed_child_pipe_fds = True
1325
1326     @contextlib.contextmanager
1327     def _on_error_fd_closer(self):
1328         """Helper to ensure file descriptors opened in _get_handles are closed"""
1329         to_close = []
1330         try:
1331             yield to_close
1332         except:
```

```
1333         if hasattr(self, '_devnull'):
1334             to_close.append(self._devnull)
1335         del self._devnull
1336     for fd in to_close:
1337         try:
1338             if _mswindows and isinstance(fd, Handle):
1339                 fd.Close()
1340             else:
1341                 os.close(fd)
1342         except OSError:
1343             pass
1344     raise
1345
1346 if _mswindows:
1347     #
1348     # Windows methods
1349     #
1350     def _get_handles(self, stdin, stdout, stderr):
1351         """Construct and return tuple with IO objects:
1352         p2cread, p2cwrite, c2pread, c2pwrite, errread, errwrite
1353         """
1354         if stdin is None and stdout is None and stderr is None:
1355             return (-1, -1, -1, -1, -1, -1)
1356
1357         p2cread, p2cwrite = -1, -1
1358         c2pread, c2pwrite = -1, -1
1359         errread, errwrite = -1, -1
1360
1361         with self._on_error_fd_closer() as err_close_fds:
1362             if stdin is None:
1363                 p2cread = _winapi.GetStdHandle(_winapi.STD_INPUT_HANDLE)
1364                 if p2cread is None:
1365                     p2cread, _ = _winapi.CreatePipe(None, 0)
1366                     p2cread = Handle(p2cread)
1367                     err_close_fds.append(p2cread)
1368                     _winapi.CloseHandle(_)
1369                 elif stdin == PIPE:
1370                     p2cread, p2cwrite = _winapi.CreatePipe(None, 0)
1371                     p2cread, p2cwrite = Handle(p2cread), Handle(p2cwrite)
1372                     err_close_fds.extend((p2cread, p2cwrite))
1373                 elif stdin == DEVNULL:
1374                     p2cread = msvcrt.get_osfhandle(self._get_devnull())
1375                 elif isinstance(stdin, int):
1376                     p2cread = msvcrt.get_osfhandle(stdin)
1377                 else:
1378                     # Assuming file-like object
1379                     p2cread = msvcrt.get_osfhandle(stdin.fileno())
1380                 p2cread = self._make_inheritable(p2cread)
1381
1382             if stdout is None:
1383                 c2pwrite = _winapi.GetStdHandle(_winapi.STD_OUTPUT_HANDLE)
1384                 if c2pwrite is None:
1385                     _, c2pwrite = _winapi.CreatePipe(None, 0)
1386                     c2pwrite = Handle(c2pwrite)
1387                     err_close_fds.append(c2pwrite)
1388                     _winapi.CloseHandle(_)
1389                 elif stdout == PIPE:
1390                     c2pread, c2pwrite = _winapi.CreatePipe(None, 0)
1391                     c2pread, c2pwrite = Handle(c2pread), Handle(c2pwrite)
1392                     err_close_fds.extend((c2pread, c2pwrite))
1393                 elif stdout == DEVNULL:
1394                     c2pwrite = msvcrt.get_osfhandle(self._get_devnull())
1395                 elif isinstance(stdout, int):
1396                     c2pwrite = msvcrt.get_osfhandle(stdout)
1397                 else:
1398                     # Assuming file-like object
1399                     c2pwrite = msvcrt.get_osfhandle(stdout.fileno())
1400                 c2pwrite = self._make_inheritable(c2pwrite)
1401
1402             if stderr is None:
1403                 errwrite = _winapi.GetStdHandle(_winapi.STD_ERROR_HANDLE)
1404                 if errwrite is None:
1405                     _, errwrite = _winapi.CreatePipe(None, 0)
1406                     errwrite = Handle(errwrite)
1407                     err_close_fds.append(errwrite)
1408                     _winapi.CloseHandle(_)
1409                 elif stderr == PIPE:
1410                     errread, errwrite = _winapi.CreatePipe(None, 0)
1411                     errread, errwrite = Handle(errread), Handle(errwrite)
1412                     err_close_fds.extend((errread, errwrite))
1413                 elif stderr == STDOUT:
1414                     errwrite = c2pwrite
1415                 elif stderr == DEVNULL:
1416                     errwrite = msvcrt.get_osfhandle(self._get_devnull())
1417                 elif isinstance(stderr, int):
1418                     errwrite = msvcrt.get_osfhandle(stderr)
1419                 else:
1420                     # Assuming file-like object
1421                     errwrite = msvcrt.get_osfhandle(stderr.fileno())
1422                 errwrite = self._make_inheritable(errwrite)
1423
```

```

1424         return (p2cread, p2cwrite,
1425                 c2pread, c2pwrite,
1426                 errread, errwrite)
1427
1428
1429     def _make_inheritable(self, handle):
1430         """Return a duplicate of handle, which is inheritable"""
1431         h = _winapi.DuplicateHandle(
1432             _winapi.GetCurrentProcess(), handle,
1433             _winapi.GetCurrentProcess(), 0, 1,
1434             _winapi.DUPLICATE_SAME_ACCESS)
1435         return Handle(h)
1436
1437
1438     def _filter_handle_list(self, handle_list):
1439         """Filter out console handles that can't be used
1440         in lpAttributeList["handle_list"] and make sure the list
1441         isn't empty. This also removes duplicate handles."""
1442         # An handle with it's lowest two bits set might be a special console
1443         # handle that if passed in lpAttributeList["handle_list"], will
1444         # cause it to fail.
1445         return list({handle for handle in handle_list
1446                     if handle & 0x3 != 0x3
1447                     or _winapi.GetFileType(handle) !=
1448                        _winapi.FILE_TYPE_CHAR})
1449
1450
1451     def _execute_child(self, args, executable, preexec_fn, close_fds,
1452                       pass_fds, cwd, env,
1453                       startupinfo, creationflags, shell,
1454                       p2cread, p2cwrite,
1455                       c2pread, c2pwrite,
1456                       errread, errwrite,
1457                       unused_restore_signals,
1458                       unused_gid, unused_gids, unused_uid,
1459                       unused_umask,
1460                       unused_start_new_session, unused_process_group):
1461         """Execute program (MS Windows version)"""
1462
1463         assert not pass_fds, "pass_fds not supported on Windows."
1464
1465         if isinstance(args, str):
1466             pass
1467         elif isinstance(args, bytes):
1468             if shell:
1469                 raise TypeError('bytes args is not allowed on Windows')
1470             args = list2cmdline([args])
1471         elif isinstance(args, os.PathLike):
1472             if shell:
1473                 raise TypeError('path-like args is not allowed when '
1474                                'shell is true')
1475             args = list2cmdline([args])
1476         else:
1477             args = list2cmdline(args)
1478
1479         if executable is not None:
1480             executable = os.fsdecode(executable)
1481
1482         # Process startup details
1483         if startupinfo is None:
1484             startupinfo = STARTUPINFO()
1485         else:
1486             # bpo-34044: Copy STARTUPINFO since it is modified above,
1487             # so the caller can reuse it multiple times.
1488             startupinfo = startupinfo.copy()
1489
1490         use_std_handles = -1 not in (p2cread, c2pwrite, errwrite)
1491         if use_std_handles:
1492             startupinfo.dwFlags |= _winapi.STARTF_USESTDHANDLES
1493             startupinfo.hStdInput = p2cread
1494             startupinfo.hStdOutput = c2pwrite
1495             startupinfo.hStdError = errwrite
1496
1497         attribute_list = startupinfo.lpAttributeList
1498         have_handle_list = bool(attribute_list and
1499                                "handle_list" in attribute_list and
1500                                attribute_list["handle_list"])
1501
1502         # If we were given an handle_list or need to create one
1503         if have_handle_list or (use_std_handles and close_fds):
1504             if attribute_list is None:
1505                 attribute_list = startupinfo.lpAttributeList = {}
1506             handle_list = attribute_list["handle_list"] = \
1507                 list(attribute_list.get("handle_list", []))
1508
1509             if use_std_handles:
1510                 handle_list += [int(p2cread), int(c2pwrite), int(errwrite)]
1511
1512             handle_list[:] = self._filter_handle_list(handle_list)
1513

```

```
1514         if handle_list:
1515             if not close_fds:
1516                 warnings.warn("startupinfo.lpAttributeList['handle_list']"
1517                               "overriding close_fds", RuntimeWarning)
1518
1519             # When using the handle_list we always request to inherit
1520             # handles but the only handles that will be inherited are
1521             # the ones in the handle_list
1522             close_fds = False
1523
1524     if shell:
1525         startupinfo.dwFlags |= _winapi.STARTF_USESHOWWINDOW
1526         startupinfo.wShowWindow = _winapi.SW_HIDE
1527         if not executable:
1528             # gh-101283: without a fully-qualified path, before Windows
1529             # checks the system directories, it first looks in the
1530             # application directory, and also the current directory if
1531             # NeedCurrentDirectoryForExePathW(ExeName) is true, so try
1532             # to avoid executing unqualified "cmd.exe".
1533             comspec = os.environ.get('ComSpec')
1534             if not comspec:
1535                 system_root = os.environ.get('SystemRoot', '')
1536                 comspec = os.path.join(system_root, 'System32', 'cmd.exe')
1537                 if not os.path.isabs(comspec):
1538                     raise FileNotFoundError('shell not found: neither %ComSpec% nor %SystemRoot% is set')
1539             if os.path.isabs(comspec):
1540                 executable = comspec
1541         else:
1542             comspec = executable
1543
1544         args = '{} /c {}'.format(comspec, args)
1545
1546     if cwd is not None:
1547         cwd = os.fsdecode(cwd)
1548
1549     sys.audit("subprocess.Popen", executable, args, cwd, env)
1550
1551     # Start the process
1552     try:
1553         hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
1554                                                  # no special security
1555                                                  None, None,
1556                                                  int(not close_fds),
1557                                                  creationflags,
1558                                                  env,
1559                                                  cwd,
1560                                                  startupinfo)
1561     finally:
1562         # Child is launched. Close the parent's copy of those pipe
1563         # handles that only the child should have open. You need
1564         # to make sure that no handles to the write end of the
1565         # output pipe are maintained in this process or else the
1566         # pipe will not close when the child process exits and the
1567         # ReadFile will hang.
1568         self._close_pipe_fds(p2cread, p2cwrite,
1569                              c2pread, c2pwrite,
1570                              errread, errwrite)
1571
1572     # Retain the process handle, but close the thread handle
1573     self._child_created = True
1574     self._handle = Handle(hp)
1575     self.pid = pid
1576     _winapi.CloseHandle(ht)
1577
1578     def _internal_poll(self, _deadstate=None,
1579                       _WaitForSingleObject=_winapi.WaitForSingleObject,
1580                       _WAIT_OBJECT_0=_winapi.WAIT_OBJECT_0,
1581                       _GetExitCodeProcess=_winapi.GetExitCodeProcess):
1582         """Check if child process has terminated. Returns returncode
1583         attribute.
1584
1585         This method is called by __del__, so it can only refer to objects
1586         in its local scope.
1587
1588         """
1589         if self.returncode is None:
1590             if _WaitForSingleObject(self._handle, 0) == _WAIT_OBJECT_0:
1591                 self.returncode = _GetExitCodeProcess(self._handle)
1592         return self.returncode
1593
1594     def _wait(self, timeout):
1595         """Internal implementation of wait() on Windows."""
1596         if timeout is None:
1597             timeout_millis = _winapi.INFINITE
1598         elif timeout <= 0:
1599             timeout_millis = 0
1600         else:
1601             timeout_millis = int(timeout * 1000)
1602         if self.returncode is None:
1603             # API note: Returns immediately if timeout_millis == 0
```

```

1604         # API Note: Returns immediately if timeout_millis == 0.
1605         result = _winapi.WaitForSingleObject(self._handle,
1606                                             timeout_millis)
1607         if result == _winapi.WAIT_TIMEOUT:
1608             raise TimeoutExpired(self.args, timeout)
1609         self.returncode = _winapi.GetExitCodeProcess(self._handle)
1610     return self.returncode
1611
1612
1613     def _readerthread(self, fh, buffer):
1614         buffer.append(fh.read())
1615         fh.close()
1616
1617
1618     def _writerthread(self, input):
1619         self._stdin_write(input)
1620
1621
1622     def _communicate(self, input, endtime, orig_timeout):
1623         # Start reader threads feeding into a list hanging off of this
1624         # object, unless they've already been started.
1625         if self.stdout and not hasattr(self, "_stdout_buff"):
1626             self._stdout_buff = []
1627             self.stdout_thread = \
1628                 threading.Thread(target=self._readerthread,
1629                                 args=(self.stdout, self._stdout_buff))
1630             self.stdout_thread.daemon = True
1631             self.stdout_thread.start()
1632         if self.stderr and not hasattr(self, "_stderr_buff"):
1633             self._stderr_buff = []
1634             self.stderr_thread = \
1635                 threading.Thread(target=self._readerthread,
1636                                 args=(self.stderr, self._stderr_buff))
1637             self.stderr_thread.daemon = True
1638             self.stderr_thread.start()
1639
1640         # Start writer thread to send input to stdin, unless already
1641         # started. The thread writes input and closes stdin when done,
1642         # or continues in the background on timeout.
1643         if self.stdin and not hasattr(self, "_stdin_thread"):
1644             self._stdin_thread = \
1645                 threading.Thread(target=self._writerthread,
1646                                 args=(input,))
1647             self._stdin_thread.daemon = True
1648             self._stdin_thread.start()
1649
1650         # Wait for the writer thread, or time out. If we time out, the
1651         # thread remains writing and the fd left open in case the user
1652         # calls communicate again.
1653         if hasattr(self, "_stdin_thread"):
1654             self._stdin_thread.join(self._remaining_time(endtime))
1655             if self._stdin_thread.is_alive():
1656                 raise TimeoutExpired(self.args, orig_timeout)
1657
1658         # Wait for the reader threads, or time out. If we time out, the
1659         # threads remain reading and the fds left open in case the user
1660         # calls communicate again.
1661         if self.stdout is not None:
1662             self.stdout_thread.join(self._remaining_time(endtime))
1663             if self.stdout_thread.is_alive():
1664                 raise TimeoutExpired(self.args, orig_timeout)
1665         if self.stderr is not None:
1666             self.stderr_thread.join(self._remaining_time(endtime))
1667             if self.stderr_thread.is_alive():
1668                 raise TimeoutExpired(self.args, orig_timeout)
1669
1670         # Collect the output from and close both pipes, now that we know
1671         # both have been read successfully.
1672         stdout = None
1673         stderr = None
1674         if self.stdout:
1675             stdout = self._stdout_buff
1676             self.stdout.close()
1677         if self.stderr:
1678             stderr = self._stderr_buff
1679             self.stderr.close()
1680
1681         # All data exchanged. Translate lists into strings.
1682         stdout = stdout[0] if stdout else None
1683         stderr = stderr[0] if stderr else None
1684
1685         return (stdout, stderr)
1686
1687     def send_signal(self, sig):
1688         """Send a signal to the process."""
1689         # Don't signal a process that we know has already died.
1690         if self.returncode is not None:
1691             return
1692         if sig == signal.SIGTERM:
1693             self.terminate()
1694         elif sig == signal.CTRL_C_EVENT:

```

```
1695         os.kill(self.pid, signal.CTRL_C_EVENT)
1696     elif sig == signal.CTRL_BREAK_EVENT:
1697         os.kill(self.pid, signal.CTRL_BREAK_EVENT)
1698     else:
1699         raise ValueError("Unsupported signal: {}".format(sig))
1700
1701     def terminate(self):
1702         """Terminates the process."""
1703         # Don't terminate a process that we know has already died.
1704         if self.returncode is not None:
1705             return
1706         try:
1707             _winapi.TerminateProcess(self._handle, 1)
1708         except PermissionError:
1709             # ERROR_ACCESS_DENIED (winerror 5) is received when the
1710             # process already died.
1711             rc = _winapi.GetExitCodeProcess(self._handle)
1712             if rc == _winapi.STILL_ACTIVE:
1713                 raise
1714             self.returncode = rc
1715
1716     kill = terminate
1717
1718     else:
1719         #
1720         # POSIX methods
1721         #
1722     def _get_handles(self, stdin, stdout, stderr):
1723         """Construct and return tuple with IO objects:
1724         p2cread, p2cwrite, c2pread, c2pwrite, errread, errwrite
1725         """
1726         p2cread, p2cwrite = -1, -1
1727         c2pread, c2pwrite = -1, -1
1728         errread, errwrite = -1, -1
1729
1730         with self._on_error_fd_closer() as err_close_fds:
1731             if stdin is None:
1732                 pass
1733             elif stdin == PIPE:
1734                 p2cread, p2cwrite = os.pipe()
1735                 err_close_fds.extend((p2cread, p2cwrite))
1736                 if self.pipesize > 0 and hasattr(fcntl, "F_SETPIPE_SZ"):
1737                     fcntl.fcntl(p2cwrite, fcntl.F_SETPIPE_SZ, self.pipesize)
1738             elif stdin == DEVNULL:
1739                 p2cread = self._get_devnull()
1740             elif isinstance(stdin, int):
1741                 p2cread = stdin
1742             else:
1743                 # Assuming file-like object
1744                 p2cread = stdin.fileno()
1745
1746             if stdout is None:
1747                 pass
1748             elif stdout == PIPE:
1749                 c2pread, c2pwrite = os.pipe()
1750                 err_close_fds.extend((c2pread, c2pwrite))
1751                 if self.pipesize > 0 and hasattr(fcntl, "F_SETPIPE_SZ"):
1752                     fcntl.fcntl(c2pwrite, fcntl.F_SETPIPE_SZ, self.pipesize)
1753             elif stdout == DEVNULL:
1754                 c2pwrite = self._get_devnull()
1755             elif isinstance(stdout, int):
1756                 c2pwrite = stdout
1757             else:
1758                 # Assuming file-like object
1759                 c2pwrite = stdout.fileno()
1760
1761             if stderr is None:
1762                 pass
1763             elif stderr == PIPE:
1764                 errread, errwrite = os.pipe()
1765                 err_close_fds.extend((errread, errwrite))
1766                 if self.pipesize > 0 and hasattr(fcntl, "F_SETPIPE_SZ"):
1767                     fcntl.fcntl(errwrite, fcntl.F_SETPIPE_SZ, self.pipesize)
1768             elif stderr == STDOUT:
1769                 if c2pwrite != -1:
1770                     errwrite = c2pwrite
1771                 else: # child's stdout is not set, use parent's stdout
1772                     errwrite = sys.__stdout__.fileno()
1773             elif stderr == DEVNULL:
1774                 errwrite = self._get_devnull()
1775             elif isinstance(stderr, int):
1776                 errwrite = stderr
1777             else:
1778                 # Assuming file-like object
1779                 errwrite = stderr.fileno()
1780
1781         return (p2cread, p2cwrite,
1782                c2pread, c2pwrite,
1783                errread, errwrite)
```

```
1785
1786 ✓ def _posix_spawn(self, args, executable, env, restore_signals, close_fds,
1787                 p2cread, p2cwrite,
1788                 c2pread, c2pwrite,
1789                 errread, errwrite):
1790     """Execute program using os.posix_spawn()."""
1791     kwargs = {}
1792     if restore_signals:
1793         # See _Py_RestoreSignals() in Python/pylifecycle.c
1794         sigset = []
1795         for signame in ('SIGPIPE', 'SIGXFZ', 'SIGXFSZ'):
1796             signum = getattr(signal, signame, None)
1797             if signum is not None:
1798                 sigset.append(signum)
1799         kwargs['setsigdef'] = sigset
1800
1801     file_actions = []
1802     for fd in (p2cwrite, c2pread, errread):
1803         if fd != -1:
1804             file_actions.append((os.POSIX_SPAWN_CLOSE, fd))
1805     for fd, fd2 in (
1806         (p2cread, 0),
1807         (c2pwrite, 1),
1808         (errwrite, 2),
1809     ):
1810         if fd != -1:
1811             file_actions.append((os.POSIX_SPAWN_DUP2, fd, fd2))
1812
1813     if close_fds:
1814         file_actions.append((os.POSIX_SPAWN_CLOSEFROM, 3))
1815
1816     if file_actions:
1817         kwargs['file_actions'] = file_actions
1818
1819     self.pid = os.posix_spawn(executable, args, env, **kwargs)
1820     self._child_created = True
1821
1822     self._close_pipe_fds(p2cread, p2cwrite,
1823                          c2pread, c2pwrite,
1824                          errread, errwrite)
1825
1826 ✓ def _execute_child(self, args, executable, preexec_fn, close_fds,
1827                   pass_fds, cwd, env,
1828                   startupinfo, creationflags, shell,
1829                   p2cread, p2cwrite,
1830                   c2pread, c2pwrite,
1831                   errread, errwrite,
1832                   restore_signals,
1833                   gid, gids, uid, umask,
1834                   start_new_session, process_group):
1835     """Execute program (POSIX version)"""
1836
1837     if isinstance(args, (str, bytes)):
1838         args = [args]
1839     elif isinstance(args, os.PathLike):
1840         if shell:
1841             raise TypeError('path-like args is not allowed when '
1842                             'shell is true')
1843         args = [args]
1844     else:
1845         args = list(args)
1846
1847     if shell:
1848         # On Android the default shell is at '/system/bin/sh'.
1849         unix_shell = ('/system/bin/sh' if
1850                      hasattr(sys, 'getandroidapilevel') else '/bin/sh')
1851         args = [unix_shell, "-c"] + args
1852     if executable:
1853         args[0] = executable
1854
1855     if executable is None:
1856         executable = args[0]
1857
1858     sys.audit("subprocess.Popen", executable, args, cwd, env)
1859
1860     if (_USE_POSIX_SPAWN
1861         and os.path.isdir(executable)
1862         and preexec_fn is None
1863         and (not close_fds or _HAVE_POSIX_SPAWN_CLOSEFROM)
1864         and not pass_fds
1865         and cwd is None
1866         and (p2cread == -1 or p2cread > 2)
1867         and (c2pwrite == -1 or c2pwrite > 2)
1868         and (errwrite == -1 or errwrite > 2)
1869         and not start_new_session
1870         and process_group == -1
1871         and gid is None
1872         and gids is None
1873         and uid is None
1874         and umask < 0):
1875         self._posix_spawn(args, executable, env, restore_signals, close_fds,
```

```
1876         p2cread, p2cwrite,
1877         c2pread, c2pwrite,
1878         errread, errwrite)
1879
1880         return
1881
1882     orig_executable = executable
1883
1884     # For transferring possible exec failure from child to parent.
1885     # Data format: "exception name:hex errno:description"
1886     # Pickle is not used; it is complex and involves memory allocation.
1887     errpipe_read, errpipe_write = os.pipe()
1888     # errpipe_write must not be in the standard io 0, 1, or 2 fd range.
1889     low_fds_to_close = []
1890     while errpipe_write < 3:
1891         low_fds_to_close.append(errpipe_write)
1892         errpipe_write = os.dup(errpipe_write)
1893     for low_fd in low_fds_to_close:
1894         os.close(low_fd)
1895     try:
1896         try:
1897             # We must avoid complex work that could involve
1898             # malloc or free in the child process to avoid
1899             # potential deadlocks, thus we do all this here.
1900             # and pass it to fork_exec()
1901
1902             if env is not None:
1903                 env_list = []
1904                 for k, v in env.items():
1905                     k = os.fsencode(k)
1906                     if b'=' in k:
1907                         raise ValueError("illegal environment variable name")
1908                     env_list.append(k + b'=' + os.fsencode(v))
1909             else:
1910                 env_list = None # Use execv instead of execve.
1911             executable = os.fsencode(executable)
1912             if os.path.isdir(executable):
1913                 executable_list = (executable,)
1914             else:
1915                 # This matches the behavior of os._execvpe().
1916                 executable_list = tuple(
1917                     os.path.join(os.fsencode(dir), executable)
1918                     for dir in os.get_exec_path(env))
1919             fds_to_keep = set(pass_fds)
1920             fds_to_keep.add(errpipe_write)
1921             self.pid = _fork_exec(
1922                 args, executable_list,
1923                 close_fds, tuple(sorted(map(int, fds_to_keep))),
1924                 cwd, env_list,
1925                 p2cread, p2cwrite, c2pread, c2pwrite,
1926                 errread, errwrite,
1927                 errpipe_read, errpipe_write,
1928                 restore_signals, start_new_session,
1929                 process_group, gid, gids, uid, umask,
1930                 preexec_fn)
1931             self._child_created = True
1932         finally:
1933             # be sure the FD is closed no matter what
1934             os.close(errpipe_write)
1935
1936     self._close_pipe_fds(p2cread, p2cwrite,
1937                         c2pread, c2pwrite,
1938                         errread, errwrite)
1939
1940     # Wait for exec to fail or succeed; possibly raising an
1941     # exception (limited in size)
1942     errpipe_data = bytearray()
1943     while True:
1944         part = os.read(errpipe_read, 50000)
1945         errpipe_data += part
1946         if not part or len(errpipe_data) > 50000:
1947             break
1948     finally:
1949         # be sure the FD is closed no matter what
1950         os.close(errpipe_read)
1951
1952     if errpipe_data:
1953         try:
1954             pid, sts = os.waitpid(self.pid, 0)
1955             if pid == self.pid:
1956                 self._handle_exitstatus(sts)
1957             else:
1958                 self.returncode = sys.maxsize
1959         except ChildProcessError:
1960             pass
1961
1962     try:
1963         exception_name, hex_errno, err_msg = (
1964             errpipe_data.split(b':', 2))
1965         # The encoding here should match the encoding
```

```
1966         # like _posixsubprocess
1967         err_msg = err_msg.decode()
1968     except ValueError:
1969         exception_name = b'SubprocessError'
1970         hex_errno = b'0'
1971         err_msg = 'Bad exception data from child: {!r}'.format(
1972             bytes(errpipe_data))
1973     child_exception_type = getattr(
1974         builtins, exception_name.decode('ascii'),
1975         SubprocessError)
1976     if issubclass(child_exception_type, OSError) and hex_errno:
1977         errno_num = int(hex_errno, 16)
1978         if err_msg == "noexec:chdir":
1979             err_msg = ""
1980             # The error must be from chdir(cwd).
1981             err_filename = cwd
1982         elif err_msg == "noexec":
1983             err_msg = ""
1984             err_filename = None
1985         else:
1986             err_filename = orig_executable
1987         if errno_num != 0:
1988             err_msg = os.strerror(errno_num)
1989         if err_filename is not None:
1990             raise child_exception_type(errno_num, err_msg, err_filename)
1991         else:
1992             raise child_exception_type(errno_num, err_msg)
1993     raise child_exception_type(err_msg)
1994
1995
1996     def _handle_exitstatus(self, sts, _del_safe=_del_safe):
1997         """All callers to this function MUST hold self._waitpid_lock."""
1998         # This method is called (indirectly) by __del__, so it cannot
1999         # refer to anything outside of its local scope.
2000         if _del_safe.WIFSTOPPED(sts):
2001             self.returncode = -_del_safe.WSTOPSIG(sts)
2002         else:
2003             self.returncode = _del_safe.waitstatus_to_exitcode(sts)
2004
2005     def _internal_poll(self, _deadstate=None, _del_safe=_del_safe):
2006         """Check if child process has terminated.  Returns returncode
2007         attribute.
2008
2009         This method is called by __del__, so it cannot reference anything
2010         outside of the local scope (nor can any methods it calls).
2011
2012         """
2013         if self.returncode is None:
2014             if not self._waitpid_lock.acquire(False):
2015                 # Something else is busy calling waitpid.  Don't allow two
2016                 # at once.  We know nothing yet.
2017                 return None
2018             try:
2019                 if self.returncode is not None:
2020                     return self.returncode # Another thread waited.
2021                 pid, sts = _del_safe.waitpid(self.pid, _del_safe.WNOHANG)
2022                 if pid == self.pid:
2023                     self._handle_exitstatus(sts)
2024             except OSError as e:
2025                 if _deadstate is not None:
2026                     self.returncode = _deadstate
2027                 elif e.errno == _del_safe.ECHILD:
2028                     # This happens if SIGCLD is set to be ignored or
2029                     # waiting for child processes has otherwise been
2030                     # disabled for our process.  This child is dead, we
2031                     # can't get the status.
2032                     # http://bugs.python.org/issue15756
2033                     self.returncode = 0
2034             finally:
2035                 self._waitpid_lock.release()
2036         return self.returncode
2037
2038
2039     def _try_wait(self, wait_flags):
2040         """All callers to this function MUST hold self._waitpid_lock."""
2041         try:
2042             (pid, sts) = os.waitpid(self.pid, wait_flags)
2043         except ChildProcessError:
2044             # This happens if SIGCLD is set to be ignored or waiting
2045             # for child processes has otherwise been disabled for our
2046             # process.  This child is dead, we can't get the status.
2047             pid = self.pid
2048             sts = 0
2049         return (pid, sts)
2050
2051
2052     def _wait(self, timeout):
2053         """Internal implementation of wait() on POSIX."""
2054         if self.returncode is not None:
2055             return self.returncode
2056
```

```
2057 if timeout is not None:
2058     endtime = _time() + timeout
2059     # Enter a busy loop if we have a timeout. This busy loop was
2060     # cribbed from Lib/threading.py in Thread.wait() at r71065.
2061     delay = 0.0005 # 500 us -> initial delay of 1 ms
2062     while True:
2063         if self._waitpid_lock.acquire(False):
2064             try:
2065                 if self.returncode is not None:
2066                     break # Another thread waited.
2067                 (pid, sts) = self._try_wait(os.WNOHANG)
2068                 assert pid == self.pid or pid == 0
2069                 if pid == self.pid:
2070                     self._handle_exitstatus(sts)
2071                     break
2072             finally:
2073                 self._waitpid_lock.release()
2074             remaining = self._remaining_time(endtime)
2075             if remaining <= 0:
2076                 raise TimeoutExpired(self.args, timeout)
2077             delay = min(delay * 2, remaining, .05)
2078             time.sleep(delay)
2079     else:
2080         while self.returncode is None:
2081             with self._waitpid_lock:
2082                 if self.returncode is not None:
2083                     break # Another thread waited.
2084                 (pid, sts) = self._try_wait(0)
2085                 # Check the pid and loop as waitpid has been known to
2086                 # return 0 even without WNOHANG in odd situations.
2087                 # http://bugs.python.org/issue14396.
2088                 if pid == self.pid:
2089                     self._handle_exitstatus(sts)
2090     return self.returncode
2091
2092
2093  def _communicate(self, input, endtime, orig_timeout):
2094     if self.stdin and not self._communication_started:
2095         # Flush stdio buffer. This might block, if the user has
2096         # been writing to .stdin in an uncontrolled fashion.
2097         try:
2098             self.stdin.flush()
2099         except BrokenPipeError:
2100             pass # communicate() must ignore BrokenPipeError.
2101         except ValueError:
2102             # ignore ValueError: I/O operation on closed file.
2103             if not self.stdin.closed:
2104                 raise
2105     if not input:
2106         try:
2107             self.stdin.close()
2108         except BrokenPipeError:
2109             pass # communicate() must ignore BrokenPipeError.
2110
2111     stdout = None
2112     stderr = None
2113
2114     # Only create this mapping if we haven't already.
2115     if not self._communication_started:
2116         self._fileobj2output = {}
2117         if self.stdout:
2118             self._fileobj2output[self.stdout] = []
2119         if self.stderr:
2120             self._fileobj2output[self.stderr] = []
2121
2122     if self.stdout:
2123         stdout = self._fileobj2output[self.stdout]
2124     if self.stderr:
2125         stderr = self._fileobj2output[self.stderr]
2126
2127     self._save_input(input)
2128
2129     if self._input:
2130         if not isinstance(self._input, memoryview):
2131             input_view = memoryview(self._input)
2132         else:
2133             input_view = self._input.cast("b") # byte input required
2134
2135     with _PopenSelector() as selector:
2136         if self.stdin and not self.stdin.closed and self._input:
2137             selector.register(self.stdin, selectors.EVENT_WRITE)
2138         if self.stdout and not self.stdout.closed:
2139             selector.register(self.stdout, selectors.EVENT_READ)
2140         if self.stderr and not self.stderr.closed:
2141             selector.register(self.stderr, selectors.EVENT_READ)
2142
2143         while selector.get_map():
2144             timeout = self._remaining_time(endtime)
2145             if timeout is not None and timeout <= 0:
2146                 self._check_timeout(endtime, orig_timeout,
```

```

2147         stdout, stderr,
2148         skip_check_and_raise=True)
2149         raise RuntimeError( # Impossible :)
2150         '_check_timeout(..., skip_check_and_raise=True) '
2151         'failed to raise TimeoutExpired.')
2152
2153     ready = selector.select(timeout)
2154     self._check_timeout(endtime, orig_timeout, stdout, stderr)
2155
2156     # XXX Rewrite these to use non-blocking I/O on the file
2157     # objects; they are no longer using C stdio!
2158
2159     for key, events in ready:
2160         if key.fileobj is self.stdin:
2161             chunk = input_view[self._input_offset :
2162                               self._input_offset + _PIPE_BUF]
2163             try:
2164                 self._input_offset += os.write(key.fd, chunk)
2165             except BrokenPipeError:
2166                 selector.unregister(key.fileobj)
2167                 key.fileobj.close()
2168             else:
2169                 if self._input_offset >= len(input_view):
2170                     selector.unregister(key.fileobj)
2171                     key.fileobj.close()
2172             elif key.fileobj in (self.stdout, self.stderr):
2173                 data = os.read(key.fd, 32768)
2174                 if not data:
2175                     selector.unregister(key.fileobj)
2176                     key.fileobj.close()
2177                 self._fileobj2output[key.fileobj].append(data)
2178     try:
2179         self.wait(timeout=self._remaining_time(endtime))
2180     except TimeoutExpired as exc:
2181         exc.timeout = orig_timeout
2182         raise
2183
2184     # All data exchanged. Translate lists into strings.
2185     if stdout is not None:
2186         stdout = b''.join(stdout)
2187     if stderr is not None:
2188         stderr = b''.join(stderr)
2189
2190     # Translate newlines, if requested.
2191     # This also turns bytes into strings.
2192     if self.text_mode:
2193         if stdout is not None:
2194             stdout = self._translate_newlines(stdout,
2195                                              self.stdout.encoding,
2196                                              self.stdout.errors)
2197         if stderr is not None:
2198             stderr = self._translate_newlines(stderr,
2199                                              self.stderr.encoding,
2200                                              self.stderr.errors)
2201
2202     return (stdout, stderr)
2203
2204
2205     def _save_input(self, input):
2206         # This method is called from the _communicate_with_*() methods
2207         # so that if we time out while communicating, we can continue
2208         # sending input if we retry.
2209         if self.stdin and self._input is None:
2210             self._input_offset = 0
2211             self._input = input
2212             if input is not None and self.text_mode:
2213                 self._input = self._input.encode(self.stdin.encoding,
2214                                                  self.stdin.errors)
2215
2216
2217     def send_signal(self, sig):
2218         """Send a signal to the process."""
2219         # bpo-38630: Polling reduces the risk of sending a signal to the
2220         # wrong process if the process completed, the Popen.returncode
2221         # attribute is still None, and the pid has been reassigned
2222         # (recycled) to a new different process. This race condition can
2223         # happens in two cases.
2224         #
2225         # Case 1. Thread A calls Popen.poll(), thread B calls
2226         # Popen.send_signal(). In thread A, waitpid() succeed and returns
2227         # the exit status. Thread B calls kill() because poll() in thread A
2228         # did not set returncode yet. Calling poll() in thread B prevents
2229         # the race condition thanks to Popen._waitpid_lock.
2230         #
2231         # Case 2. waitpid(pid, 0) has been called directly, without
2232         # using Popen methods: returncode is still None is this case.
2233         # Calling Popen.poll() will set returncode to a default value,
2234         # since waitpid() fails with ProcessLookupError.
2235         self.poll()
2236         if self.returncode is not None:
2237             # Skip signalling a process that we know has already died.

```

```
2237         # Skip signalling process that we know has already died.
2238         return
2239
2240     # The race condition can still happen if the race condition
2241     # described above happens between the returncode test
2242     # and the kill() call.
2243     try:
2244         os.kill(self.pid, sig)
2245     except ProcessLookupError:
2246         # Suppress the race condition error; bpo-40550.
2247         pass
2248
2249     def terminate(self):
2250         """Terminate the process with SIGTERM
2251         """
2252         self.send_signal(signal.SIGTERM)
2253
2254     def kill(self):
2255         """Kill the process with SIGKILL
2256         """
2257         self.send_signal(signal.SIGKILL)
```